

Übergabe - Weiterentwicklung "Montagefortschritt"

Stand: 2026-06-15

Da du die **Ursprungs-App** gebaut hast (Supabase-basiert), ist diese Übergabe als **Delta** gedacht: was sich gegenüber deiner Version geändert hat und wo die neuen Bausteine im Code sitzen. Das Datenmodell der Positionen kennst du bereits - neu sind v. a. das **Backend (Microsoft 365 statt Supabase)**, die **Offline-PWA-Schicht** und ein paar Fachfeatures. Vertiefende Doku: - ZUSAMMENFASSUNG.md - Funktionsumfang & Historie - SICHERHEIT.md - Auth, Berechtigungen, Least-Privilege-Setup - SETUP.md / m365-setup.md - Microsoft-365-Einrichtung

0. Was sich gegenüber der Ursprungs-App (Supabase) geändert hat

Thema	Ursprungs-App (deine)	Jetzt
Backend	Supabase REST (/rest/v1/<table>, anon-Key)	Microsoft Graph + SharePoint
Auth	anon-Key im Code	MSAL-Login (KPC-M365-Konto, delegiert)
Fortschritt	1 Supabase-Zeile je Pos (g/m/a/ib/ew/note/fotos)	SharePoint-Liste "Positionen", 1 Item je Pos
Fotos	Base64 in der Tabelle	echte Dateien in Bibliothek "Fotos", in Ordnern Baustelle/Etage/Anlage
Stückliste	eingebettetes const DATA	externe LV_<pid>.json je Projekt (in "Fotos")
Felder	g, m, a, ib, ew, note, fotos	+ sa (Sanitär-Anschluss), ea (Elektro-Anschluss), by (Ausführer je Haken), ts (Zeitstempel)
Betrieb	online	PWA, voll offline (IndexedDB + Outbox-Sync)
Projekte	i. d. R. eine App pro Baustelle	Mehrprojekt + Stücklisten-Import-Assistent
Sonstiges	-	Gewerk-Filter M/S/E, Firma/Monteur-Zuordnung, Hell/Dunkel, Corporate Design

Deine alten Tools sind nach LV_<pid>.json + SharePoint **migriert** worden (Berlin Upbeat, Mensa GGS). Die Migrationsfunktionen sind inzwischen wieder aus dem Code entfernt (siehe SICHERHEIT.md).

1. Was ist die App?

Eine **Web-App zur Leistungsfeststellung / Montagefortschritt** auf Küchen-Baustellen. Pro Baustelle gibt es eine Stückliste (das LV) mit Positionen. Monteure haken pro Position ab: **Geliefert, Montiert, Sanitär-Anschluss, Elektro-Anschluss, Inbetriebnahme, Einweisung**, machen **Fotos** und erzeugen **PDF-Protokolle**. Jeder Haken wird mit **Zeitstempel und Ausführer** (eigener Monteur oder Fremdfirma) festgehalten. Die App ist eine **PWA**: installierbar, voll **offline-fähig** mit automatischer Synchronisierung, sobald wieder Netz da ist.

2. Technischer Aufbau (bewusst einfach!)

Es gibt kein Build-System, kein npm, kein Framework. Die ganze App ist eine einzige HTML-Datei mit eingebettetem CSS und JavaScript. Bearbeiten = Datei editieren. Veröffentlichen = Datei hochladen. Das ist Absicht - maximal wartbar ohne Toolchain.

Deploy-Dateien (nur diese 3 landen auf dem Server):

Datei	Inhalt
index.html	Die komplette App (~3000 Zeilen: HTML + CSS + JS)
sw.js	Service Worker (Offline-Cache der App-Shell)
manifest.webmanifest	PWA-Manifest (Name, Icons, Farben)

Alles andere im Ordner (* .md) ist nur Doku und wird **nicht** deployt.

Hosting: GitHub Pages, Repo kpcgmbh/Montagefortschritt, Live-URL

https://kpcgmbh.github.io/Montagefortschritt/ (Groß-/Kleinschreibung beachten). Externe Bibliotheken (PDF.js, jsPDF, JSZip, MSAL, SheetJS/xlsx) kommen per CDN - siehe <script src=...>-Tags in index.html und die Liste in sw.js.

Backend: Es gibt **keinen eigenen Server**. Daten liegen in **Microsoft 365 / SharePoint**

(kpcfulda.sharepoint.com/sites/Montagefortschritt): - Liste "**Positionen**" = der Fortschritt (Häkchen, Zeitstempel, Ausfühler, Notizen) - Bibliothek "**Fotos**" = Fotos, PDF-Protokolle und die LV-Dateien (LV_<pid>.json)

Zugriff per **Microsoft Graph API**, Anmeldung per **MSAL** (Microsoft-Login). Details/Berechtigungen -> SICHERHEIT.md.

3. Aufbau von index.html - wo finde ich was?

Grobe Landkarte (Zeilennummern wandern bei Änderungen, daher zur Orientierung besser nach den **Kommentar-Überschriften** suchen):

Bereich	Marker / Funktionen
PWA-Registrierung, Schrift	<!-- PWA / Offline -->, <!-- Corporate-Design-Schrift -->
Offline / Sync-Engine	OFFLINE / PWA - IndexedDB-Persistenz, Outbox ... - enqueuePos, flushOutbox, spListWriteFields
Login-Stufen (HTML)	<!-- Stufe 1: Microsoft-Anmeldung -->, <!-- Stufe 2 -->
Anmelde-Logik	onLoggedIn()
Backend M365	BACKEND: Microsoft 365 ... - getToken, spListLoadRows, spListWrite, spUpload, spDownloadBlob
Gewerk-Filter (M/S/E)	parseGewerk, setTF, applyF
Ausfühler (Firma/Monteur)	gtsAltbestand, Badge-Logik
Fotos in Ordner	bilderEinordnen, folderFor, ensureFolder
PDF-Editor / Protokolle	PROTOKOLL UPLOAD + PDF EDITOR, PDF EDITOR (Pencil)
Mehrprojekt + Import	MEHRPROJEKT + STÜCKLISTEN-IMPORT - loadProjects, loadProjectLV

Datenmodell LV (LV_<pid>.json): Array von Sektionen

```
[{ pos, bez, label, subsections:[
  { pos, bez, items:[
    { pos, menge, einh, bez, fabrikat, typ, gew:[...] } ] } ] ] ]
```

- pos = Positionsnummer (z. B. 01.01.00.00.05.), eindeutiger Schlüssel.
- gew = Gewerk-Array, Werte "M" (Montage), "S" (Sanitär), "E" (Elektro). Steuert den Ansicht-Umschalter/Filter in der Liste.
- **Wichtig:** Der **Fortschritt** (Häkchen etc.) steht NICHT in der LV-Datei, sondern getrennt in der SharePoint-Liste "Positionen". Darum kann man das LV gefahrlos ergänzen, ohne Häkchen zu verlieren (siehe §6).

4. Lokal bearbeiten & testen (Windows)

Befehle unten sind **PowerShell** (Arbeitsplatz Windows). Editor egal - die App ist eine einzige HTML-Datei ohne Build-Schritt.

Schneller Syntaxcheck der Inline-Skripte vor dem Upload, falls Node vorhanden:

```
node -e "const h=require('fs').readFileSync('index.html','utf8'); const s=[...h.matchAll(/<script(?:![^>]*\bsrc=)[^>]*>([\s\S]*?)<\script>/g)].map(m=>m[1]).join('\n;\n'); new Function(s); console.log('PARSE OK');"
```

PARSE OK = nur Syntax ok, sagt nichts über Logik. Alternativ reicht die **F12-Konsole** beim lokalen Öffnen.

Echt testen geht nur **online + eingeloggt** (Login/Daten laufen live über Microsoft, lokal nicht). Datei hochladen (§5), auf der Live-URL testen - am besten in einem **InPrivate-Fenster** (Strg+Umschalt+N), damit kein alter Service-Worker-Cache stört. Beim Debuggen des Caches: DevTools -> **Application -> Service Workers -> Unregister**.

5. Veröffentlichen (Deploy)

1. Die 3 Dateien (meist nur index.html) ins GitHub-Repo kpcgmbh/Montagefortschritt pushen (oder über die GitHub-Weboberfläche committen). GitHub Pages deployt automatisch.
2. **Wenn du sw.js oder die CDN-Bibliotheksliste geändert hast:** in sw.js die Zeile const CACHE = 'kpc-montage-vN' **hochzählen** (z. B. v8 -> v9). Sonst behalten die Geräte den alten Cache. (Reine index.html-Änderungen brauchen das nicht - die wird "network-first" immer frisch geladen.)
3. Auf dem Gerät einmal neu laden. Bei zähem Cache: hart neu laden (**Strg+F5** bzw. **Strg+Umschalt+R**).

Verifizieren, dass live = lokal (PowerShell im Projektordner):

```
$cb = [DateTimeOffset]::Now.ToUnixTimeSeconds()  
Invoke-WebRequest "https://kpcgmbh.github.io/Montagefortschritt/index.html?cb=$cb"  
-OutFile "$env:TEMP\live.html"  
(Get-FileHash index.html -Algorithm SHA256).Hash  
(Get-FileHash "$env:TEMP\live.html" -Algorithm SHA256).Hash # beide Hashes muessen  
gleich sein
```

6. Zusammenarbeit im Team (Git statt Ordnerfreigabe)

Gemeinsamer Arbeitsbereich ist das GitHub-Repo - nicht ein geteilter OneDrive-Ordner. Zwei Gründe:
1. OneDrive führt parallele Änderungen an einer Datei **nicht** zusammen - es entstehen "...-Konflikt-Kopien..." oder eine Version überschreibt die andere. Bei der einen großen index.html ist das ein echtes Datenverlust-Risiko. Git merged zeilenweise und meldet nur bei *derselben* Zeile einen Konflikt. 2. GitHub Pages liefert die App aus dem **Repo** aus, nicht aus OneDrive. Ein geteilter Ordner ändert die Live-App also gar nicht - es muss ohnehin alles durch GitHub.

Aufteilung: - **Code** (index.html, sw.js, manifest.webmanifest) -> GitHub-Repo - **Doku, Stücklisten, Pläne, Fotos** -> gerne weiter OneDrive (dort ist es richtig)

Ablauf je Änderung: 1. git pull - aktuellen Stand holen, nie auf veralteter Datei arbeiten. 2. Branch anlegen (z. B. feature/gewerk-status), **nicht** direkt auf main (main = live). 3. Ändern, lokal/Preview testen (§4). 4. **Pull Request** -> der/die andere schaut kurz drüber -> Merge nach main. 5. GitHub Pages deployt automatisch.

Hinweise: - Den Git-Klon **nicht** in den OneDrive-Ordner legen - OneDrive und der versteckte .git-Ordner vertragen sich schlecht. Lieber z. B. C:\Projekte\Montagefortschritt. - Kleine, unkritische Fixes dürfen direkt auf main - aber **immer vorher git pull**. - Die Änderungshistorie steckt in **Commits + Pull Requests** (aussagekräftige Beschreibung). Das ist das Änderungsprotokoll - **keine neue MD-Datei pro Änderung**. Bestehende Docs (ZUSAMMENFASSUNG.md, SICHERHEIT.md, diese Datei) bei Bedarf aktualisieren.

7. Daten/Projekte - sichere Vorgehensweise

Neues Projekt anlegen (frische Baustelle, noch kein Fortschritt): In der App **" + Neue Baustelle aus Stückliste "** -> Excel hochladen -> Projektname vergeben. Der Import-Assistent baut daraus das LV. (Excel-Format: eine Zeile je Position mit Spalten u. a. Pos-Nr, Menge, Einheit, Bezeichnung, Fabrikat, Typ, **Gewerk** = M/S/E, mit + kombinierbar.)

Bestehendes Projekt ändern (z. B. Gewerk nachpflegen): NIEMALS als "neue Baustelle" neu importieren - das würde die Verknüpfung zum Fortschritt sprengen. Stattdessen die **LV_<pid>.json in der Fotos-Bibliothek direkt ersetzen** (gleiche pid behalten). Der Fortschritt in der Liste "Positionen" bleibt unberührt, weil er über die pos verknüpft ist.

Bereits angebundene Projekte: - berlin-upbeat - Berlin Upbeat (388 Positionen, alle Gewerke gesetzt) - mensa-ggs - Mensa GGS Bergisch Gladbach (aus Alt-Tool migriert) - Göttingen Kita Montessori 261038 - in Vorbereitung (Stückliste-Entwurf liegt in Montagefortschritt1/Kita Montessori/)

8. Sicherheit - das Wichtigste in Kürze

- Anmeldung nur mit **KPC-Microsoft-Konto** (Single-Tenant). Kein Passwort-Handling in der App, **keine Geheimnisse im Code** (Client-/Tenant-ID sind öffentlich und ok).
 - Die App hat **minimale Rechte**: User.Read + Sites.Selected, freigeschaltet **nur** für die Montagefortschritt-Site. Andere SharePoint-Sites sind tabu.
 - Der Quellcode ist öffentlich (GitHub Pages), aber **ohne gültiges Konto wertlos** - die Daten schützt der Microsoft-Login + die SharePoint-Berechtigungen.
 - **Achtung beim Scopes-Ändern:** Wenn du in index.html GRAPH_SCOPES erweiterst, musst du die passende Berechtigung auch in der **Azure-App-Registrierung** hinzufügen + Admin-Zustimmung geben - sonst gibt es 403. Ablauf steht in SICHERHEIT.md.
-

9. Konventionen & Fallstricke

- **Sprache:** UI und Code-Kommentare auf **Deutsch**. Schrift durchgängig **Roboto Condensed Light** (nicht ändern ohne Designabsprache).
 - **Hell/Dunkel:** automatisch nach Gerätehelligkeit + manueller Umschalter (data-theme). CSS nutzt semantische Variablen (--page/--surface/--text/--line/ --accent). Keine festen Farben hartcodieren - immer die Variablen verwenden.
 - **Offline:** Schreibzugriffe gehen erst in die **Outbox** (IndexedDB) und werden bei "online" gesynct. Nie direkt synchron schreiben und das Offline-Modell umgehen.
 - **SharePoint-Dateinamen:** unzulässige Zeichen vorher mit safeName() ersetzen; Pfade pro Segment mit encPath() kodieren.
 - **Graph/Login/SharePoint werden NIE gecacht** (network-only im Service Worker) - diese Regel in sw.js nicht aufweichen, sonst riskiert man veraltete Daten/Tokens.
 - Vor jedem Upload: PARSE OK-Check (§4) und nach dem Upload Hash-Abgleich (§5).
-

10. Typische Aufgaben - Kurzrezept

- **Gewerk eines Projekts ergänzen:** LV-JSON anpassen -> LV_<pid>.json in Fotos- Bibliothek ersetzen. Fertig.
- **Neues Feld pro Position:** in index.html Render + Save erweitern; das SharePoint-Liste-Feld in "Positionen" anlegen (Spalte) und in spListWrite/ spListLoadRows mappen.
- **Neue Bibliothek/CDN einbinden:** <script src>-Tag in index.html + Eintrag in SHELL in sw.js + Cache-Version hochzählen.
- **Design anpassen:** CSS-Variablen im :root-Block bzw. Dark-Block ändern.

Bei Unsicherheit: erst auf einem Test-Projekt / im Inkognito-Fenster ausprobieren, nie direkt auf einer Produktiv-Baustelle mit echten Häkchen.